

UNIVERSIDAD NACIONAL AUTÓNOMA
DE MÉXICO

FACULTAD DE CIENCIAS

MODELADO Y PROGRAMACIÓN

DOCUMENTACIÓN

PRÁCTICA 4:
PRUEBAS UNITARIAS

DISEÑO DE PRUEBAS PARA UN SISTEMA

P R E S E N T A N :

INTEGRANTES DEL EQUIPO:

Alan Gael Alvarez Hernández
María Fernanda Miranda Sánchez
Pedro Pablo Ruiz Hernández

PROFESORA:
Karla Adriana Esquivel Guzmán

AYUDANTE DE TEORÍA:
Ossiel Aguilar Espíndola

AYUDANTE DE LABORATORIO:
Enoc Pineda Aguirre



FACULTAD DE CIENCIAS
UNAM

SEMESTRE 1-2027

Ciudad de México, 20 de octubre de 2026

Índice

1. Introducción **3**

1.1. Pruebas Unitarias 3

1.2. Estructura de una prueba Arrange - Act - Asseert 3

2. Objetivo **4**

3. Desarrollo de la práctica **4**

3.1. Durante la práctica 4

3.2. Plantemaiento del Problema 5

3.3. Planteamiento de Pruebas 6

3.4. Implementación en Python 6

4. Checklist **6**

Introducción

Pruebas Unitarias

Una **Prueba Unitaria** verifica un comportamiento pequeño del sistema: una función, un método o una regla concreta. Para el desarrollo de las mismas se recomienda seguir el siguiente flujo:

$$\textbf{Problema} \longrightarrow \textbf{Modelo} \longrightarrow \textbf{Casos} \longrightarrow \textbf{Pruebas} \longleftrightarrow \textbf{Implementación}$$

Figura 1: Flujo general del diseño de pruebas.

Nota: El resultado esperado se decide antes de ejecutar el código. No se copia lo que devuelve la implementación.

Los casos de prueba definidos se clasifican como:

- **Normal:** Un valor dentro de la regla.
- **Frontera:** Justo antes, en y después de un límite.
- **Inválido:** Un valor que el sistema debe rechazar.
- **Excepción:** Un error esperado.
- **Cruce de reglas:** Cuando dos condiciones se cumplen al mismo tiempo.

Ejemplo:

Si hasta 15 minutos es gratis, y desde 16 se cobra, probar 14, 15, 16 y 17.

Estructura de una prueba Arrange - Act - Asseert

- **Arrange:** Preparación de los datos, objetos, o estado inicial.
- **Act**: Ejecución de la función o método que se desea probar.
- **Assert:** Comprobación de que lo obtenido sea lo esperado.

Ejemplo:

```
# Arrange
minutos = 16

# Act
resultado = sistema.calcular_total(minutos, "normal", False)

# Assert
assert resultado == 20.0
```

Nota: Una buena prueba responde tres preguntas: **¿qué regla comprueba?, ¿qué entrada utiliza? y ¿qué resultado debería producir?**

Objetivo

El alumno diseñará un conjunto de pruebas unitarias para verificar el comportamiento de un sistema a partir de su especificación, no únicamente a partir de lo que hace el código.

Al finalizar la práctica, el alumno deberá ser capaz de:

- identificar casos normales;
- identificar casos frontera;
- detectar entradas inválidas;
- verificar errores y excepciones;
- escribir pruebas usando el patrón Arrange–Act–Assert;
- justificar de qué regla del modelo proviene cada caso de prueba;
- ejecutar una suite de pruebas y analizar sus resultados;
- diferenciar entre “la prueba pasó” y “el programa es correcto”.

Desarrollo de la práctica

Durante la práctica

1. Lea primero el modelo.
2. Diseñe los casos sin ejecutar todavía el sistema.
3. Escriba el resultado esperado de cada caso.
4. Implemente las pruebas.
5. Ejecute la suite.
6. Si alguna prueba falla, no cambie inmediatamente la prueba.
7. Compare:

modelo esperado vs. **resultado obtenido**

1. Determine si el problema esta en:
   a) la implementación;
   b) la prueba;
   c) la interpretación del modelo.

Plantemaiento del Problema

Probar el sistema de Ticket de estacionamiento.

El sistema recibe:

- minutos de permanencia;
- tipo de cliente;
- si perdió el boleto;

Modelo del Sistema: Ticket de estacionamiento

Comportamiento esperado:

R1. Los minutos deben ser un número entero mayor o igual que 0.

R2. Los tipos de clientes válidos son:

- **"normal"**
- **"frecuente"**

R3. Si el boleto está perdido, el costo final es de $300.00, sin importar los minutos ni el tipo de cliente.

R4. Si los minutos son 0, el costo es $0.00

R5. De 1 a 15 minutos, el costo es $0.00

R6. De 16 a 60 minutos, el costo es de $20.00

R7. Después de 60 minutos se cobran $20.00 por la primera hora y $15.00 por cada hora adicional **iniciada**.

R8. Un cliente **"frecuente"** recibe un 10 % de descuento sobre el costo calculado, excepto cuando perdió el boleto.

R9. El costo nunca puede ser negativo.

R10. Si los minutos son negativos, el sistema debe lanzar un error.

R11. Si el tipo de cliente no es válido, el sistema debe lanzar un error.

Ejemplos del Sistema: Ticket de estacionamiento

Ejemplos de interpretación:

- 15 minutos, **cliente normal** → $0.00
- 16 minutos, **cliente normal** → $20.00
- 60 minutos, **cliente normal** → $20.00

- 61 minutos, **cliente normal** → $35.00
- 120 minutos, **cliente normal** → $35.00
- 121 minutos, **cliente normal** → $50.00
- 61 minutos, **cliente frecuente** → $20.00
- *boleto perdido* → $300.00

Planteamiento de Pruebas

Implementación en Python

Checklist

Ejemplo:

- ☒ Actividad terminada
- ⊟ Actividad parcialmente realizada
- ☐ Una actividad

Actividades:

- ⊟ Documentación.
- ☐ Planteamiento de Pruebas.
- ☐ Implementación de Pruebas.
- ☐ Evidencia de Ejecución.
- ☐ Preguntas de Análisis.
- ☐ Organización de Archivos.
- ☐ README.md.